FEATURE & UX ROADMAP

ZoneRuns

A grounded punch list of what to add or fix next — built by auditing every screen, component, and service in the current codebase, not generic app-idea brainstorming. Every item below references the real file or function it touches.

Prepared for Filip Sinjur · 9 November 2026 · ZoneRuns (Expo / React Native / Supabase + PostGIS)

BUG something's broken today FEATURE net-new capability UX polish on what exists

TECH foundation work Effort: S / M / L — rough gut-feel, not a quote

Since your report

Your strokovno poročilo named four concrete follow-ups in its conclusion. One is already done. The other three are carried forward into Technical Foundations below, with credit.

- ✓ **Done —** viewport-based territory loading. The report asked for loading territories "glede na vidno polje karte (onBoundsChanged)" for scalability; `territory-map.tsx` now reports map bounds on `moveend` and the parent re-queries `get_territories_in_bounds` per viewport instead of fetching everything.
- Still open — polyline/distance smoothing reconciliation → Technical Foundations, item 16
- Still open — offline queue's narrow network-error matching → Technical Foundations, item 17
- Still open — map tile caching for offline use → Technical Foundations, item 18

1 Quick Wins

BUG Effort: S

1. Fix avatar upload on Android

`profileService.uploadAvatar` still uploads via `fetch(uri).blob()`. `socialService.uploadPostImage`'s own code comment explains this exact pattern breaks Supabase Storage uploads on Android, and already works around it using `expo-file-system`'s `File#arrayBuffer()`. Port the same fix to avatars — same bug, one function.

BUG Effort: S

2. Fix chat & group-chat history

`getMessages` and `getGroupMessages` both run `.order('created_at', {ascending:true}).limit(100)`. Past 100 messages, a thread shows its *oldest* messages, not its most recent — long conversations will look broken or stuck. Switch to descending + limit and reverse client-side, or move to real keyset pagination while you're in there (pairs with item 12).

FEATURE Effort: M

3. Build an Achievements screen

The backend is already fully built: 29 seeded achievements across six categories, `getAchievementsByCategory`, `getUserAchievements`, `getAchievementProgress`, and even an `ACHIEVEMENT_ICONS` icon map all exist and work — and surface nowhere. Today the only feedback a user gets is a single `Alert.alert` right after saving a run. A badge gallery with progress bars is close to pure front-end work for one of the biggest engagement wins on this entire list.

UX Effort: S

4. Clean up dead screens

`app/explore.tsx` and `app/modal.tsx` are unmodified Expo template leftovers that nothing in the app links to. `react-native-maps` is an unused dependency now that every map renders through Leaflet-in-WebView. Free cleanup, smaller bundle.

UX Effort: M

5. Add a first-run explainer for loop-closing

The core hook of the app — run a loop that closes within ~40m of where it started to claim the enclosed area — is explained nowhere in-app. There's no onboarding flow at all (`profileService.hasProfile()` even exists for gating one and is never called). A short carousel, or even a single tooltip over the Compete button the first time someone opens it, would cut a lot of "wait, how does this work?" confusion.

UX Effort: S

6. Add accessibility labels

Zero `accessibilityLabel` / `accessibilityRole` usages exist anywhere in the app today. Start with icon-only controls — back buttons, delete/share icons, header actions — where a screen reader currently has nothing to announce.

2 Core Gamification Depth

FEATURE Effort: M

7. Individual leaderboards

The only leaderboard that exists, `get_group_leaderboard`, ranks public groups by summed member territory — there's no personal ranking anywhere. Add a global leaderboard and a friends-only one (by distance, territory km^2, or streak), reusing the `followers` table you already have for the friends filter.

FEATURE Effort: M

8. Tell people when their territory gets stolen

`claim_territory_from_run` already computes exactly who lost area to whom (`stolen_from` in `territory_claims`), but only the thief ever sees a result — a one-shot `Alert`. The victim finds out only if they happen to open the map. Surface it to them too: in-app at minimum, push once item 11 exists.

FEATURE Effort: L

9. Territory decay / re-contest

Right now the map only ever fills in — nothing is ever released back. An optional rule where territory unclaimed-for-N-days goes neutral (or becomes easier to re-steal) keeps the board alive and gives returning players something to reclaim instead of a map that's eventually just solved.

UX Effort: S

10. Show achievements on profiles

`user-profile.tsx` (viewing someone else) shows stats but zero badges, and your own Profile tab doesn't show them either. Natural follow-on once item 3 exists — and a good hook for other people to see what they're missing.

3 Social & Communication

FEATURE Effort: L

11. Push notifications

Currently zero infrastructure: no `expo-notifications` dependency, no device-token table, no Supabase Edge Functions at all in the project. Start with local notifications (weekly-goal reminder, "your streak is about to break") — these need no backend — then add a token table plus one minimal Edge Function for the social triggers: new follower, new message, territory stolen.

UX Effort: M

12. Chat polish

Add infinite-scroll pagination for older messages (pairs with fixing item 2), a typing indicator, and a read-receipt mark for the sender. None of these exist in `chat.tsx` or `group-chat.tsx` today — both are realtime-insert-only with a flat 100-message initial load.

4 Personalization & Settings

FEATURE Effort: M

13. A real Settings screen

Everything settings-adjacent currently lives inline in the Profile tab. Pull it into its own screen: unit system (distance is hardcoded to km everywhere right now), notification preferences once item 11 exists, account deletion, an about/version line.

BUG Effort: M

14. Wire up privacy settings — or remove them

`profile_settings` (`profile_visible`, `stats_visible`, `runs_visible`, `achievements_visible`) exists in the database and is entirely decorative: not checked by any RLS policy, not read by any screen. Either enforce it properly end-to-end (RLS + client) or drop the facade — right now it would let a future Settings screen promise privacy it doesn't deliver.

UX Effort: M

15. Settle the language

Screens are a hardcoded mix of Slovenian (most of the app) and English (`messages.tsx`, `chat.tsx`), with no i18n library anywhere. Pick one consistently, or add a lightweight setup (`expo-localization` + a strings file) and get bilingual SI/EN close to free — a nice touch if this project ever gets shown beyond class.

5 Technical Foundations

TECH Effort: M

16. Reconcile polyline smoothing with distance filtering

From your original report's future-work list, still open: the drawn route includes every raw GPS point while the distance calculation filters out >100m jumps as GPS noise, so the visible path is noticeably less smooth than the computed distance implies.

TECH Effort: S

17. Broaden offline-queue error detection

Also from the report, and verified still true in `runsService.ts` today: a failed save only queues for offline retry if the error message literally contains `"network"` or `"fetch"`. Other failure types — a strict RLS 403, for instance — fall through, and the run is silently lost instead of queued.

TECH Effort: M

18. Cache map tiles for offline use

The report's third open item. OSM/CARTO tiles aren't cached anywhere, so the map renders blank without a connection even though a run itself can still be recorded and queued offline.

TECH Effort: M

19. Finish the offline queue

`offlineQueue.ts` declares action types for `UPDATE_RUN`, `DELETE_RUN`, `CREATE_POST`, `LIKE_POST`, and `UNLIKE_POST`, but only `CREATE_RUN` actually has a handler registered. The others silently no-op when offline today — either implement them or stop declaring them.

TECH Effort: S

20. Stop recomputing stats from scratch

`updateUserStatsAfterRun` refetches and re-sums *every* historical run, on every single save, to rebuild `user_stats`. Fine at today's scale, won't stay fine — move to incrementally updating the row, or compute it in Postgres instead of in JS.

Appendix — smaller inconsistencies

- Three screens (`profile.tsx`, `signin.tsx`, `verify.tsx`) hand-roll a breathing-ring animation with the legacy `Animated` API instead of reusing the Reanimated `BreathingIcon` component that already exists for exactly this.
- `getRecentRuns`, `searchProfiles`, and `searchPublicGroups` have no pagination — a flat `.limit()` only. Fine today, worth remembering as lists grow.
- No block, report, or mute tooling exists anywhere — worth having before any wider rollout beyond friends/classmates.
- Avatar upload and post-image upload use two different code paths for what should be the same operation (see item 1) — worth unifying into one shared upload helper once fixed.